ENTRIX — EXHAUSTIVE ENGINEERING IMPLEMENTATION PLAN

AI-Powered Integrated Enterprise Resource Planning Platform

Document Reference: ENTRIX-IMP-V1.0
Target Duration: 10 Weeks (5 Phased Sprints)
Academic Cohort: Ramdeobaba University, B.Tech. CSE Sem V (2026–27)
Project Members: Aditya Sakarkar (A1-16), Chetan Gupta (A1-50), Pranav Rathi (A1-52)
Target Architecture: Monorepo (`client/` , `server/` , `ai-service/` , `deploy/`)

1. Master Technology & Tool Mapping Matrix

This matrix maps every technology committed in **Section 6 of the Project Synopsis** to its exact repository location and operational responsibility:

Domain	Technology / Library	Repository Location	Exact Responsibility in ENTRIX
Frontend Core	React.js 18 + Vite	`client/`	Fast SPA rendering, component modularity, Hot Module Reloading
Type Safety	TypeScript	`client/` & `server/`	Compile-time strict interface typing, preventing runtime NPEs
Styling	Tailwind CSS	`client/src/index.css`	Utility-first responsive design system, KPI cards, modal dialogs
State Management	Redux Toolkit (RTK)	`client/src/store/`	Centralized global store for auth token, live notifications, cart/PR
Real-time Client	Socket.io-client	`client/src/services/`	WebSocket listener for approval alerts, inventory drops, fraud flags
Backend Core	Node.js v20 LTS + Express.js	`server/src/app.ts`	REST API gateway, route orchestration, middleware pipelines
Authentication	Passport.js + JWT	`server/src/auth/`	Bearer token validation, local strategy, password hashing
Database ORM	Sequelize / TypeORM	`server/src/models/`	PostgreSQL relational mapping, migrations, seeders, transactions
Primary Database	PostgreSQL 16	External Docker / Cloud	ACID relational storage, B-Tree indexes, UUID primary keys
Cache & Session	Redis 7	External Docker / Cloud	Token blacklisting, dashboard metric cache, rate-limit counters
File Storage	Amazon S3 SDK	`server/src/utils/s3.ts`	Secure pre-signed uploads for invoices, receipts, and user avatars
AI Framework	Python 3.11 + FastAPI	`ai-service/main.py`	Asynchronous REST microservice for numerical and ML inference
Machine Learning	Scikit-learn, Pandas, NumPy	`ai-service/models/`	Random Forest demand regression, Isolation Forest anomaly detection
DevOps / Cloud	Docker + Docker Compose	Root `/`	Multi-container local virtualization, standardized deployments
Cloud Target	AWS (EC2, S3, CloudWatch)	Cloud Infrastructure	Production cloud hosting, serverless storage, centralized logs
CI/CD	GitHub Actions	`.github/workflows/`	Automated linting, test suites, and Docker image build on push
Testing Suite	Jest + React Testing Library	`server/tests/`, `client/tests/`	Unit and integration test coverage for state machine and UI components

Domain	Technology / Library	Repository Location	Exact Responsibility in ENTRIX
Code Quality	ESLint + Prettier + SonarQube	Root configs	Static code analysis, security vulnerability scanning, code format

2. Directory Layout & Module Structure

```
d:\E-comm\
├── .github/
│   └── workflows/
│       ├── backend-ci.yml            # Lint, test, and build server
│       ├── frontend-ci.yml           # Lint, type-check, test client
│       └── ai-service-ci.yml         # Flake8, pytest for Python service
├── client/                           # React.js Presentation Tier
│   ├── public/
│   ├── src/
│   │   ├── assets/
│   │   ├── components/               # Reusable UI primitives (Buttons, Modals, Tables)
│   │   │   ├── common/
│   │   │   ├── forms/
│   │   │   └── navigation/
│   │   ├── layouts/                  # AppLayout, AuthLayout, DashboardLayout
│   │   ├── modules/                  # Feature-based modular pages
│   │   │   ├── auth/                 # Login, Password Reset
│   │   │   ├── hr/                   # Directory, Attendance, Workload
│   │   │   ├── inventory/            # Stock Master, Low Stock Alerts
│   │   │   ├── procurement/          # PR Pipeline, PO Issuer, GRN Receiver
│   │   │   ├── finance/              # Ledger, 3-Way Match, Anomaly Review
│   │   │   ├── projects/             # Tasks, Milestones, Workload
│   │   │   └── analytics/            # Executive AI Forecast & Spend Graphs
│   │   ├── services/                 # Axios HTTP clients, Socket.io client
│   │   ├── store/                    # Redux Toolkit slices (authSlice,
notificationSlice)
│   │   ├── types/                    # TypeScript shared interfaces
│   │   ├── App.tsx
│   │   └── main.tsx
│   ├── package.json
│   ├── tsconfig.json
│   ├── tailwind.config.js
│   └── vite.config.ts
├── server/                           # Node.js API Service Tier
│   ├── src/
│   │   ├── config/                   # PostgreSQL, Redis, S3, Passport configs
│   │   ├── constants/                # Roles, State Machine Enums, HTTP Codes
│   │   ├── controllers/              # HTTP Request/Response handlers
│   │   ├── middlewares/              # JWT Auth, RBAC guard, Rate limiter, ErrorHandler
│   │   ├── models/                   # Sequelize PostgreSQL relational models
│   │   │   ├── User.ts
│   │   │   ├── Role.ts
│   │   │   ├── Department.ts
```

```
│   │   │   ├── InventoryItem.ts
│   │   │   ├── PurchaseRequest.ts
│   │   │   ├── PurchaseOrder.ts
│   │   │   ├── GoodsReceivedNote.ts
│   │   │   ├── Invoice.ts
│   │   │   ├── Project.ts
│   │   │   └── AuditLog.ts
│   │   ├── routes/                    # Modular Express route definitions
│   │   ├── services/                  # Business logic (Workflow engine, S3 uploader)
│   │   ├── sockets/                   # Socket.io event emitters and rooms
│   │   ├── utils/                     # Hash chaining, Logger (Winston), Circuit breaker
│   │   └── app.ts
│   ├── migrations/                    # Sequelize database migration scripts
│   ├── seeders/                       # Demo seed records (Admin, Items, Departments)
│   ├── tests/                         # Jest API unit & integration tests
│   ├── package.json
│   └── tsconfig.json
├── ai-service/                        # Python FastAPI Machine Learning Tier
│   ├── app/
│   │   ├── api/                       # FastAPI router endpoints (/forecast, /anomaly)
│   │   ├── core/                      # Environment settings, logger
│   │   ├── models/                    # Scikit-learn serialized models (.joblib)
│   │   ├── schemas/                   # Pydantic request/response validation schemas
│   │   └── services/                  # Model training and inference logic
│   │       ├── demand_forecaster.py
│   │       ├── anomaly_detector.py
│   │       └── budget_predictor.py
│   ├── tests/                         # Pytest test suites
│   ├── requirements.txt
│   └── main.py
├── deploy/                            # Deployment & Cloud Ops
│   ├── nginx/
│   │   └── default.conf               # Reverse proxy config (Port 80/443 routing)
│   ├── docker-compose.yml             # Multi-container orchestration (local & production)
│   ├── docker-compose.dev.yml         # Dev environment overrides
│   └── Dockerfile.server
├── .env.example
├── sonar-project.properties           # SonarQube quality gate scanner configuration
└── README.md
```

3. Five-Phase Exhaustive Roadmap

PHASE 1: Architectural Foundation, Database Schema & Authentication (Weeks 1–2)

Goal: Establish a production-grade development environment, initialize PostgreSQL database schemas, and enforce JWT-based Role-Based Access Control (RBAC).

Week 1: Environment Orchestration & Data Modeling

- **Day 1**: Monorepo scaffolding. Initialize Git repository, configure `.gitignore`, ESLint, Prettier, and root `docker-compose.yml` (PostgreSQL 16, Redis 7).
- **Day 2**: Database schema design. Write Sequelize migration files for 10 tables: `departments`, `roles`, `users`, `inventory_items`, `purchase_requests`, `purchase_orders`, `goods_received_notes`, `invoices`, `projects`, `audit_logs`.
- **Day 3**: Constraints & Indexes. Define foreign keys with strict `ON DELETE RESTRICT`, check constraints (`stock_qty >= 0`, `price >= 0`), and B-Tree indexes on `users(email)`, `inventory(sku_code)`.
- **Day 4**: Database Seeders. Script realistic test datasets: 5 departments, 4 default roles (`SUPER_ADMIN`, `HR_MANAGER`, `FINANCE_OFFICER`, `PROCUREMENT_OFFICER`, `STAFF`), 20 test users, and 50 stock items.
- **Day 5**: Container validation. Verify database migration execution inside Docker containers; verify Redis connection and health checks.

Week 2: Authentication Pipeline & UI Design System

- **Day 6**: Passport.js & JWT Integration. Implement `POST /api/v1/auth/login` and `POST /api/v1/auth/refresh`. Generate salted bcrypt password hashes (10 salt rounds).
- **Day 7**: RBAC Middleware. Construct Express middleware `requireRole(['SUPER_ADMIN', 'FINANCE_OFFICER'])` with claim validation.
- **Day 8**: Frontend Scaffolding. Initialize Vite React TypeScript project. Configure Tailwind CSS typography, color palette, and layout shells (Sidebar, Navbar, Mobile drawer).
- **Day 9**: Auth State in Redux. Implement `authSlice.ts` managing user JWT, session persistence in `localStorage`, and Axios interceptors for automatic token refresh on `HTTP 401`.
- **Day 10**: Phase 1 Gate Review. Execute Jest auth unit tests; verify protected route redirection in React router (`react-router-dom v6`).

PHASE 2: Core ERP Modules — HR, Finance Ledger & Inventory Master (Weeks 3–4)

Goal: Build functional operational capabilities across Human Resources, General Financial Ledger, and Real-Time Stock Management.

Week 3: Human Resource & Employee Management

- **Day 11**: Employee Directory CRUD APIs. `GET /api/v1/hr/employees`, `POST /api/v1/hr/employees`, `PUT /api/v1/hr/employees/:id`.
- **Day 12**: Attendance & Workload Tracking. Daily status logging API (`PRESENT`, `ABSENT`, `LEAVE`); calculate departmental attendance percentage.
- **Day 13**: HR UI Screens. React data grid displaying employee profiles with search filters, department dropdowns, and status badges.
- **Day 14**: Leave Management Workflow. Employee leave request modal, manager sign-off API, and leave balance decrement logic.
- **Day 15**: Unit Testing. Jest tests for employee duplication validation and attendance date integrity.

Week 4: Inventory Master & Financial Ledger Setup

- **Day 16**: Inventory Master APIs. SKU creation, category grouping, unit pricing, safety threshold settings, and current balance tracking.
- **Day 17**: Inventory UI & Low Stock Visualizer. Responsive data grid with search, threshold alerts, and red-highlighted rows for items where `stock_qty <= safety_stock`.
- **Day 18**: Double-Entry General Ledger. Model `ledger_entries` with simultaneous debit/credit validation:

  $$\sum \text{Debits} - \sum \text{Credits} = 0$$

  $\sum$ Debits − $\sum$ Credits = 0
- **Day 19**: Department Budget Allocation. Implement quarterly budget caps in `departments`; build API to check allocated vs consumed expenses.
- **Day 20**: Phase 2 Gate Review. Verify transactional isolation during inventory adjustments; verify financial ledger zero-balance audit.

PHASE 3: Procurement State Machine, Approval Pipeline & Audit Logging (Weeks 5–6)

Goal: Build the core differentiator: the automated 8-step procurement pipeline and tamper-evident audit trail.

Week 5: Procurement Requisition Engine & Dual-Tier Approval

- **Day 21**: Requisition Submission API. `POST /api/v1/procurement/requests`. Validate quantity, calculate `estimated_total`, set state to `PENDING_MANAGER`.
- **Day 22**: State Machine Logic. Implement strict transition guards:
  - *Branch A* (`estimated_total < ₹10,000`): Manager approval advances directly to `APPROVED`.
  - *Branch B* (`estimated_total >= ₹10,000`): Manager approval advances to `PENDING_FINANCE`.
- **Day 23**: Socket.io Push Notifications. Real-time WebSocket alerts dispatched to target approver rooms upon state transition.
- **Day 24**: Procurement Pipeline UI. Interactive Kanban board visualizing PR cards moving across status columns with one-click **Approve** / **Reject** modals.
- **Day 25**: Rejection Handling. Implement mandatory rejection comments and status update to `REJECTED`.

Week 6: PO Issuance, Goods Receipt (GRN) & Cryptographic Audit Trails

- **Day 26**: Purchase Order Generation. Convert approved PR into formal PO binding supplier information, payment terms, and delivery deadlines.
- **Day 27**: Goods Received Note (GRN). Warehouse intake API: Verify physical shipment quantity, generate GRN, and increment `inventory_items.stock_qty` in an atomic PostgreSQL transaction.
- **Day 28**: 3-Way Invoice Matching. Finance verification API: Match Vendor Invoice amount against PO unit cost and GRN quantity within $\pm ₹0.01$ tolerance.
- **Day 29**: Cryptographic Audit Hash Chaining. Implement SHA-256 hash chaining on all state mutations:
  $$\text{Hash}_i = \text{SHA256}(\text{Hash}_{i-1} + \text{actor_id} + \text{action} + \text{timestamp} + \text{payload})$$
  Hashi = SHA256(Hashi−1 + actor_id + action + timestamp + payload)
- **Day 30**: Phase 3 Gate Review. Execute end-to-end simulated purchase workflow from employee requisition to stock increment and invoice clearance.

PHASE 4: Python FastAPI Microservice & Machine Learning Analytics (Weeks 7–8)

Goal: Implement predictive intelligence: Scikit-learn models for inventory demand forecasting, expense fraud detection, and project budget overrun predictions.

Week 7: FastAPI Setup & Demand Forecasting Service

- **Day 31**: FastAPI Microservice Scaffolding. Setup Python 3.11 environment, Pydantic schemas, and CORS middleware running on port 8000.

- **Day 32**: Synthetic Dataset Generator. Create Python script generating 365 days of seasonal inventory consumption and procurement expense records.
- **Day 33**: Demand Forecast Model. Train `RandomForestRegressor` on lag features (`lag_7`, `lag_14`, `rolling_mean_7`, `day_of_week`). Serialize model using `joblib`.
- **Day 34**: `/forecast/demand` API Endpoint. Expose endpoint returning 30-day predicted consumption curves and recommended reorder batch sizes.
- **Day 35**: Node.js Circuit Breaker. Implement Node.js client (`axios`) with 2,000 ms timeout and fallback to local 30-day moving averages if Python service is offline.

Week 8: Expense Anomaly Detection & Business Insights

- **Day 36**: Anomaly Detection Model. Train `IsolationForest` (contamination = 0.03) on transaction amounts, department spending variance, and invoice timestamps.
- **Day 37**: `/anomaly/detect-expense` API Endpoint. Flag invoices with decision score < -0.5 as anomalous fraud candidates.
- **Day 38**: Project Overrun Predictor. Train regression model predicting project completion cost based on milestone progress vs consumed developer hours.
- **Day 39**: Analytics Dashboard UI. Integrate Recharts in React frontend displaying interactive demand forecast bands and flagged anomaly cards.
- **Day 40**: Phase 4 Gate Review. Test ML inference latency (must complete under 1,500 ms) and verify graceful fallback behavior.

PHASE 5: Security Hardening, Testing, Dockerization & AWS Cloud Deployment (Weeks 9–10)

Goal: Hardening, automated CI/CD pipeline, comprehensive testing, Docker multi-container packaging, and AWS cloud deployment.

Week 9: Security Hardening, Testing Suites & SonarQube

- **Day 41**: Security Hardening. Apply Helmet.js secure headers, configure CORS whitelist, enforce express-rate-limit (100 req/min), validate all input via Zod/Joi.
- **Day 42**: Unit & Integration Testing. Write Jest test suites for Node.js backend achieving > 75% code coverage across critical financial and state machine services.
- **Day 43**: React Component Testing. Test React UI components using React Testing Library (login form, approval buttons, modal submissions).
- **Day 44**: Static Code Analysis. Configure SonarQube scanner (`sonar-project.properties`) and resolve static code smells, vulnerabilities, and duplicate code blocks.

- **Day 45**: Amazon S3 Storage Integration. Implement S3 bucket file storage for invoice PDF uploads and pre-signed retrieval URLs.

Week 10: Multi-Stage Dockerization, GitHub Actions & AWS Deployment

- **Day 46**: Production Dockerfiles. Author multi-stage build Dockerfiles for `client` (Nginx Alpine), `server` (Node.js Alpine), and `ai-service` (Python Slim).
- **Day 47**: GitHub Actions CI/CD. Create `.github/workflows/deploy.yml` to run tests, build Docker containers, and push images to GitHub Container Registry.
- **Day 48**: AWS EC2 & CloudWatch Deployment. Launch AWS EC2 t3.medium instance; provision Docker Compose environment; configure AWS CloudWatch log agent for centralized monitoring.
- **Day 49**: End-to-End Smoke Test. Perform complete academic project demonstration dry-run: User creation ➔ PR submission ➔ Dual approval ➔ PO ➔ GRN ➔ Inventory increase ➔ AI Forecast view.
- **Day 50**: Final Documentation & Freeze. Package deliverables, export Postman API collections, and generate final presentation slides for academic evaluation.

4. Work Breakdown by Team Member

Team Member	Roll No.	Primary Responsibilities across 10 Weeks
Aditya Sakarkar	A1-16	System Architecture, Docker orchestration, Procurement State Machine, AWS CloudWatch deployment, CI/CD pipelines.
Chetan Gupta	A1-50	Node.js Backend API, PostgreSQL relational schemas, Sequelize migrations, Passport.js/JWT auth, Jest testing.
Pranav Rathi	A1-52	Python FastAPI AI microservice, Scikit-learn forecasting & anomaly models, React.js frontend dashboards, Socket.io integration.

5. Verification & Acceptance Milestone Checklist

```
[ ] Milestone 1: PostgreSQL 16 schema running via Docker with 10 tables & foreign keys.
[ ] Milestone 2: JWT Authentication functioning with RBAC permission enforcement.
[ ] Milestone 3: Purchase Request dual-tier approval branching (< ₹10k vs >= ₹10k)
verified.
[ ] Milestone 4: Goods Received Note (GRN) atomically increments inventory stock in DB.
[ ] Milestone 5: Double-entry ledger postings balance with zero discrepancy.
[ ] Milestone 6: Cryptographic audit log hash chain validates correctly without breaks.
[ ] Milestone 7: Python FastAPI returns 30-day demand forecast within 1,500 ms SLA.
[ ] Milestone 8: Isolation Forest flags anomalous test invoices with score < -0.5.
[ ] Milestone 9: React SPA displays live Socket.io notifications upon manager approval.
[ ] Milestone 10: SonarQube static analysis passes with Zero Critical Vulnerabilities.
[ ] Milestone 11: Multi-container system boots cleanly with single `docker compose up -d`.
```

6. Risk Management & Fallback Protocols

Identified Risk	Severity	Mitigation / Fallback Architecture
Python ML Microservice Unavailability	High	Asynchronous circuit breaker in Node.js falls back to 30-day simple moving average; transactions never block.
Database Network Partition During GRN	Critical	Transactions run inside explicit `BEGIN ... COMMIT` blocks; failures execute instant rollback to prevent phantom stock.
WebSocket Connection Drops on Client	Medium	Client automatically polls fallback REST endpoint `/api/v1/notifications/unread` every 60 seconds if socket disconnects.
Cloud Hosting Budget Exhaustion	Low	Entire stack is containerized with Docker Compose; can instantly pivot from AWS to local evaluation laptop without code changes.